Universidade Federal do Cariri - UFCA

Proposta Técnica de Projeto

Programação Orientada a Objetos

Equipe:

Sebastião Sousa Soares
Espedito Ramom Mascena Ricarto
Sabrina Alencar Soares

Juazeiro do Norte - CE
Janeiro de 2026

Sumário

1 Identificação da Equipe **2**

2 Visão Geral do Projeto **2**
2.1 Descrição do Domínio 2
2.1.1 Modo Clássico Competitivo 2
2.1.2 Modo Clássico Treino 2
2.2 Justificativa da Complexidade e Escolha do Paradigma 3

3 Arquitetura e Modelagem **3**
3.1 Especificação de Hierarquias 4
3.2 Composição e Agregação 4

4 Padrões e Princípios **4**
4.1 Padrões de Projeto (Design Patterns) 4
4.2 Princípios SOLID 5

1 Identificação da Equipe

- **Sebastião Sousa Soares** (`https://github.com/sebastiaosoares`)
- **Espedito Ramom Mascena Ricarto** (`https://github.com/ramomricarto`)
- **Sabrina Alencar Soares** (`https://github.com/sabrinaalencaar`)

2 Visão Geral do Projeto

2.1 Descrição do Domínio

O projeto, denominado de **Trocadu**, consiste num jogo digital do estilo *Party Game*, feito para ser jogado em grupo usando apenas um telemóvel (passando o aparelho de mão em mão). O objetivo do sistema é controlar automaticamente as regras de uma partida de adivinhação, retirando dos jogadores a necessidade de marcar pontos ou cronometrar o tempo manualmente.

O sistema possui foco em dois modos de jogo principais:

2.1.1 Modo Clássico Competitivo

Este modo é projetado para ser jogado de 3 a 5 jogadores. A dinâmica funciona com duplas temporárias (um Descritor e um Adivinhador), onde os jogadores alternam entre todas as possibilidades existentes de formação de duplas.

- **Descritor:** Vê a palavra no ecrã (uma por vez) e tenta descrevê-la. Possui à disposição um botão para "Pular"(limitado a n vezes por turno) e um botão de "Acerto".
- **Adivinhador:** Tenta acertar a palavra.

Os pontos são somados cumulativamente ao longo das rodadas. Uma rodada completa é composta por um turno de "ida"e um de "volta". Os pontos conquistados pela dupla são creditados individualmente para cada um dos dois jogadores. No final, a classificação exibe os jogadores de forma individual com os seus pontos totais acumulados em todas as rodadas.

Neste modo, as palavras provêm de pacotes que podem ser padronizados ou personalizados. Pacotes personalizados só podem ser criados por utilizadores autenticados. É importante destacar a separação conceptual: um **Jogador** não é necessariamente um **Utilizador** com sessão iniciada, visto que uma única pessoa pode configurar a partida localmente no seu telemóvel e adicionar os nomes dos seus amigos.

2.1.2 Modo Clássico Treino

Focado na prática individual ou casual. O jogador joga sozinho (ou interage com amigos livremente) sem o rigor do modo competitivo.

- Não há assistência do contador do Trocadu, cronómetro (timer) ou limite máximo de palavras.
- Não é possível utilizar pacotes personalizados, apenas palavras aleatórias do sistema.

- As palavras surgem no ecrã e o jogador possui botões para marcar acerto ou pular indefinidamente.
- Idealizado para apenas um jogador e um telemóvel.

2.2 Justificativa da Complexidade e Escolha do Paradigma

A escolha do paradigma **Orientado a Objetos (POO)** justifica-se pela necessidade de gerir múltiplos estados simultâneos (cronómetros, pontuações de equipes temporárias e repositórios de palavras), algo que num modelo procedural resultaria em alto acoplamento. O sistema atende aos critérios de complexidade possuindo regras de negócio explícitas, decisões de fluxo (como o controlo de rodízio "Todos contra Todos") e variações claras de comportamento mediante o uso de polimorfismo central. O sistema é composto por mais de 12 classes no total, incluindo ao menos 8 de domínio (como `Palavra`, `Turno`, `Jogador`, `Equipe`, `Usuario`), além de serviços e fábricas. O **encapsulamento** protege os dados contra alterações indevidas, utilizando atributos privados e propriedades lógicas.

3 Arquitetura e Modelagem

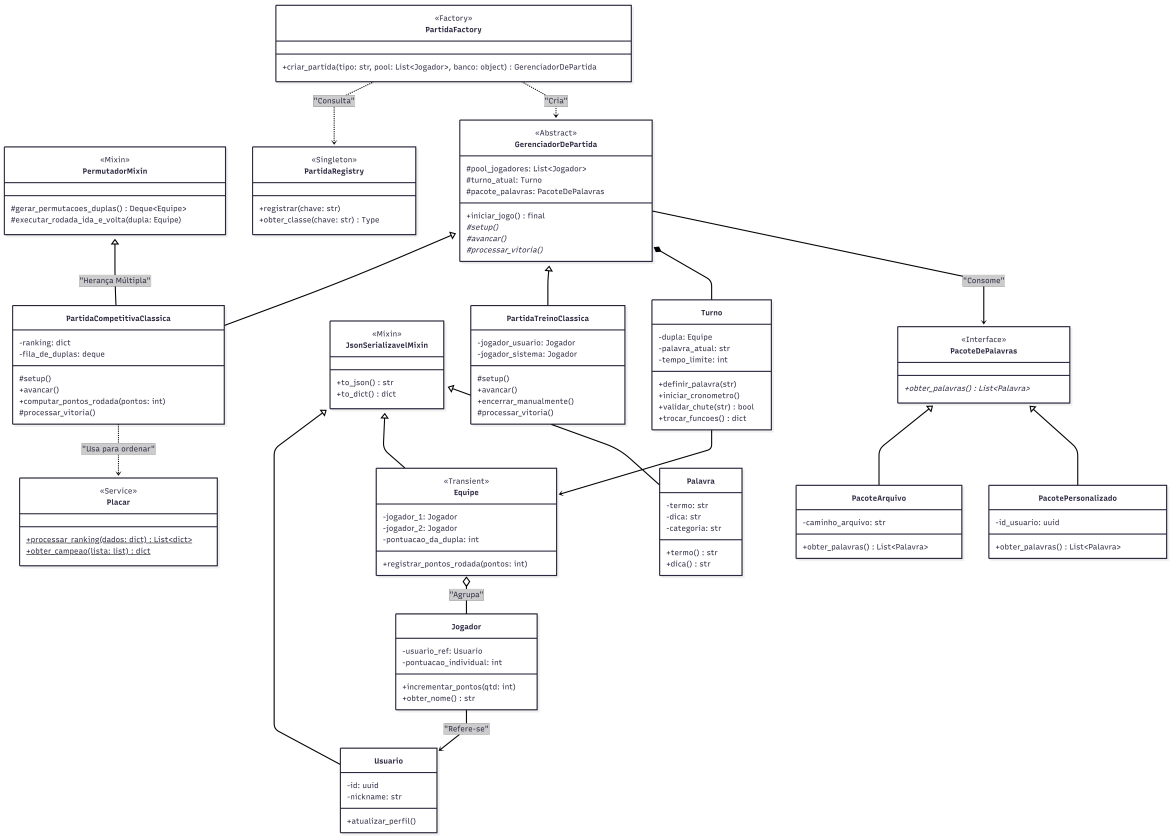

Figura 1: Diagrama de Classes do Trocadu

3.1 Especificação de Hierarquias

O projeto contempla duas hierarquias principais e o uso de herança múltipla via Mixins, atendendo aos requisitos de abstração conceptual e variação de comportamento.

1. **Hierarquia de Abstração Conceptual (Fonte de Dados)** Isola a origem das palavras da lógica de jogo.

   Classe Abstrata: `PacoteDePalavras`. Define o contrato `obter_palavras()`.

   Subclasses: `PacoteArquivo` (arquivos JSON) e `PacotePersonalizado` (base de dados/memória de utilizador autenticado). O polimorfismo permite a troca da fonte sem alterar o núcleo do jogo.

2. **Hierarquia de Variação de Comportamento (Regras do Jogo)** Altera o funcionamento do fluxo de processamento e encerramento.

   Classe Abstrata: `GerenciadorDePartida`. Utiliza *Template Method* para orquestrar o esqueleto da partida.

   Subclasses: `PartidaCompetitiva` (regras de pontuação estrita, rodízio de duplas e classificação individual) e `PartidaTreino` (loop infinito, um único jogador, sem temporizador ou contabilização na classificação).

3. **Herança Múltipla e Mixins** Utilizamos `JsonSerializavelMixin` e `PermutadorMixin` para acoplar comportamentos transversais. Entidades como `Usuario` e `Equipe` herdam o comportamento de serialização sem criar uma árvore de herança rígida e genérica.

3.2 Composição e Agregação

Cumprindo os requisitos estruturais, destacam-se as seguintes relações:

- **Composição (Ciclo de Vida Dependente):** 1. A classe `GerenciadorDePartida` compõe instâncias de `Turno`. Quando a partida é destruída, o turno ativo deixa de existir. 2. A classe `Turno` possui composição sobre os contadores de tempo e limites de saltos.

- **Agregação (Independência de Objetos):** 1. A classe `Equipe` (dupla temporária) agrega dois objetos da classe `Jogador`. A equipe deixa de existir a cada rodada, mas os jogadores continuam no sistema guardando os seus pontos individuais. 2. A classe `Jogador` agrega uma referência opcional à classe `Usuario` (para distinguir o jogador local da conta persistente do criador de pacotes).

4 Padrões e Princípios

4.1 Padrões de Projeto (Design Patterns)

Foram aplicados padrões do catálogo *Gang of Four* (GoF):

Template Method: Aplicado em `GerenciadorDePartida`. Define o esqueleto do algoritmo do jogo. As subclasses `PartidaCompetitiva` e `PartidaTreino` implementam apenas as abstrações (`setup` e `processar_vitoria`), variando entre acumular pontos em rodízio ou gerar palavras aleatórias sem tempo.

Factory Method: A classe `PartidaFactory` encapsula a lógica de criação. O cliente solicita uma partida, e a fábrica decide qual gestor retornar.

Strategy: Relação entre o jogo e a hierarquia `PacoteDePalavras`. Permite trocar a fonte de palavras dinamicamente.

4.2 Princípios SOLID

O projeto obedece aos princípios SOLID:

SRP (Single Responsibility): Responsabilidades isoladas. `GerenciadorDePartida` lida com o macro-fluxo; `Turno` apenas com os saltos e a mecânica imediata.

OCP (Open/Closed): O sistema está fechado para modificações, mas aberto à extensão, sendo fácil criar novos modos (ex: Modo Kids) estendendo o `GerenciadorDePartida`.

LSP (Liskov Substitution): As subclasses `PartidaCompetitiva` e `PartidaTreino` substituem perfeitamente o `GerenciadorDePartida` sem quebrar o sistema central.

ISP (Interface Segregation): Uso de Mixins (`JsonSerializavelMixin`) para injetar capacidades apenas onde é estritamente necessário.

DIP (Dependency Inversion): O gestor depende da interface `PacoteDePalavras`, abstraindo a implementação real.